

Contents

1.1 Propósito	2
1.2 Convenciones	2
Niveles de Prioridad	2
Identificadores de Requisitos	2
1.3 Audiencia Prevista	2
1. Equipo de Desarrollo (Ingenieros, Arquitectos, QA)	2
2. Stakeholders (Clientes, Organizadores y Negocio)	2
3. Profesorado y Evaluadores Académicos	3
1.4 Alcance	3
1. Dentro del Alcance (Incluido)	3
2. Fuera del Alcance (Excluido explícitamente)	3
1.5 Referencias	3
Estándares de Ingeniería y Arquitectura	3
Normativa Legal y Seguridad (Compliance)	4
Documentación Técnica Oficial	4
2. Descripción General	4
2.1 Perspectiva del Producto	4
2.2 Funciones del Producto	4
2.3 Clases de Usuario y Características	4
2.4 Entorno Operativo	4
2.5 Restricciones	5
2.6 Documentación de Usuario	5
2.7 Supuestos y Dependencias	5
3. Requisitos Funcionales	5
3.1 Autenticación y Autorización	5
3.2 Gestión de Eventos	5
3.3 Ticketing y Reservas (Bookings)	6
3.4 Procesamiento de Pagos	6
3.5 Validación en Puerta (Check-in)	6
3.6 Interacción Social	6
4. Interfaces Externas	6
4.1 Interfaces de Usuario (UI)	6
4.2 Interfaces de Hardware	6
4.3 Interfaces de Software (APIs y Servicios)	6
4.4 Interfaces de Comunicaciones	7
5. Requisitos No Funcionales	7
5.1 Rendimiento y Escalabilidad	7
5.2 Seguridad y Cumplimiento Legal	7
5.3 Confiabilidad y Disponibilidad	7
5.4 Usabilidad y Accesibilidad	7
5.5 Mantenibilidad e Infraestructura	7
6. Otros Requisitos y Apéndices	8
6.1 Migración de Datos e Internacionalización	8
Apéndice A: Glosario de Estados Centrales	8
Apéndice B: Lista de Problemas Pendientes / Riesgos (Risk Register)	8

1.1 Propósito

El propósito de este Documento de Especificación de Requisitos de Software (ERS) es definir clara y detalladamente los requerimientos funcionales y no funcionales para el sistema **Event Management**.

Este documento está dirigido a todo el equipo involucrado en el proyecto, incluyendo: - **Equipo Técnico (Desarrolladores)** - **Stakeholders (Clientes/Interesados)** - **Profesorado/Evaluadores**

En el día a día, la ERS será utilizada principalmente por los desarrolladores y stakeholders para alinear expectativas, guiar el desarrollo de nuevas funcionalidades y validar el software entregado.

Este documento actuará como un **documento vivo** a lo largo del ciclo de vida del proyecto. Su rol no es ser un contrato estático, sino una base fundamental y dinámica para documentar los requisitos core a medida que el proyecto evoluciona.

1.2 Convenciones

Este documento utiliza las siguientes convenciones tipográficas y de nomenclatura para asegurar la precisión y evitar ambigüedades en la interpretación de los requisitos, basándose en los lineamientos del estándar RFC 2119:

Niveles de Prioridad

- **DEBE (MUST)**: Indica que el requisito es de cumplimiento obligatorio. El sistema no será aceptable si no cumple con esta condición (Generalmente asociado a la Fase 1 / MVP).
- **DEBERÍA (SHOULD)**: Indica que el requisito es altamente recomendado, pero puede existir una razón válida en circunstancias particulares para ignorarlo (Generalmente asociado a la Fase 2).
- **PUEDE (MAY)**: Indica que el requisito es opcional o deseable, pero no crítico para el funcionamiento principal del sistema (Generalmente asociado a la Fase 3 en adelante).

Identificadores de Requisitos

Para garantizar la trazabilidad, cada requisito tendrá un identificador único y secuencial. Estos identificadores **nunca** serán reasignados, incluso si el requisito original es eliminado o descartado. - **RF-XX**: Requisito Funcional (ej. RF-01, RF-02). - **RNF-XX**: Requisito No Funcional (ej. RNF-01, RNF-02).

1.3 Audiencia Prevista

Este documento está diseñado para ser consultado y utilizado por múltiples roles dentro del ecosistema del proyecto **Event Management**. Cada perfil interactuará con el documento de manera distinta:

1. Equipo de Desarrollo (Ingenieros, Arquitectos, QA)

- **Uso**: Servirá como la “fuente de la verdad” para la implementación técnica. Guiará el desarrollo de la arquitectura basada en Hotwire, el uso de Service Objects (ej. manejo de concurrencia con bloqueos pesimistas), la integración con la pasarela de pagos (Culqi) y la generación de códigos QR. También será la base para la redacción de casos de prueba (RSpec/Capybara).

2. Stakeholders (Clientes, Organizadores y Negocio)

- **Uso**: Validar que las características propuestas resuelvan las necesidades del negocio. Les permitirá confirmar que el flujo de compra de tickets, la gestión del aforo y las capacidades de los distintos roles (Admin, Organizer, Registered User) están alineados con sus expectativas antes de su despliegue final.

3. Profesorado y Evaluadores Académicos

- **Uso:** Como instrumento de auditoría y evaluación. Permitirá verificar el cumplimiento de los estándares de la industria, la aplicación de patrones de diseño avanzados, estrategias de integración (Webhooks vs ESB) y la completitud del producto según la rúbrica del curso.

1.4 Alcance

Este sistema es una plataforma integral para la gestión de eventos, venta de entradas y control de acceso. Para evitar ambigüedades, a continuación se definen explícitamente los límites del sistema.

1. Dentro del Alcance (Incluido)

El sistema **DEBE** incluir las siguientes capacidades: - **Gestión de Eventos:** Creación (modo borrador/publicado), edición, categorización y manejo de imágenes mediante Active Storage. - **Geolocalización:** Renderizado de mapas interactivos (Leaflet) para la ubicación exacta de los eventos. - **Gestión de Entradas (Ticketing):** Control de aforo concurrente estricto utilizando bloqueos a nivel de base de datos (Pessimistic Locking) para evitar sobreventa. - **Pagos y Transacciones:** Integración real con pasarelas de pago (Culqi API v4), procesamiento de webhooks para cambios de estado, y simulador local (**MockGateway**) para pruebas. - **Autenticación y Autorización:** Registro tradicional con verificación de email por código numérico y SSO con Google OAuth 2.0. Autorización granulada por roles (Admin, Organizer, User) mediante Pundit. - **Control de Acceso (Check-in):** Generación de códigos QR para tickets e interfaz para validación en puerta. - **Interacción Social:** Sistema de comentarios y reseñas en eventos publicados. - **Integración de Ecosistema:** Capacidad de notificar/proveer datos a sistemas externos (turismo, alquileres, restaurantes) mediante Webhooks/APIs REST en tiempo diferido.

2. Fuera del Alcance (Excluido explícitamente)

El sistema **NO** incluirá las siguientes características en esta fase del proyecto: - **Streaming de Video:** No se hostearán ni se transmitirán eventos en vivo o virtuales dentro de la plataforma. - **Asignación Visual de Asientos:** No existirá un mapa interactivo de asientos (ej. plateas numeradas en teatros); la venta se limita a cantidades globales por categoría de ticket (`ticket_type`). - **Integración de Bus de Servicio Empresarial (ESB):** La integración con el resto del ecosistema de turismo/servicios se hará mediante soluciones ligeras (REST/Webhooks) y no mediante una capa ESB centralizada. - **Facturación Electrónica Automática (SUNAT/AFIP):** La plataforma procesa el pago a través de Culqi, pero no genera ni envía comprobantes de pago fiscales autorizados automáticamente. - **Chat en Vivo:** No se implementará mensajería en tiempo real entre asistentes y organizadores.

1.5 Referencias

Dado que el sistema opera como una startup real procesando pagos e información personal de usuarios (con foco inicial en el mercado peruano, inferido por la integración con Culqi y datos geográficos), la plataforma y este documento se rigen y referencian en los siguientes estándares y normativas:

Estándares de Ingeniería y Arquitectura

1. **ISO/IEC/IEEE 29148:2018** - *Systems and software engineering — Life cycle processes — Requirements engineering*. (Base formal para la estructura de esta Especificación de Requisitos).
2. **RFC 2119** - *Key words for use in RFCs to Indicate Requirement Levels*. (Define el uso estricto de las palabras DEBE, DEBERÍA, PUEDE en la sección 1.2).

Normativa Legal y Seguridad (Compliance)

3. **Ley N° 29733 (Perú)** - *Ley de Protección de Datos Personales*. (Mandatoria para el tratamiento, almacenamiento y transferencia de los correos electrónicos, nombres y datos de los asistentes/organizadores).
4. **PCI DSS (Payment Card Industry Data Security Standard)** - *SAQ A (Self-Assessment Questionnaire A)*. (Aplica a startups que tercerizan el procesamiento de tarjetas a pasarelas como Culqi, exigiendo que la web cumpla con estándares estrictos de seguridad y cifrado TLS 1.3).

Documentación Técnica Oficial

5. **Documentación Oficial API Culqi v4** - <https://apidocs.culqi.com/> (Referencia técnica para las implementaciones de los webhooks y la tokenización en la pasarela de pagos).
6. **Ruby on Rails 8 Guides** - <https://guides.rubyonrails.org/> (Referencia para convenciones del framework, Hotwire y Solid Queue).

2. Descripción General

2.1 Perspectiva del Producto

Event Management es una plataforma web independiente, pero diseñada bajo una arquitectura orientada a la integración. Actúa como el origen de la verdad (fuente principal) para la creación de eventos y venta de entradas, y se proyecta como el núcleo de un ecosistema más amplio (que incluirá plataformas de turismo, alquileres y restaurantes) con las cuales se comunicará primariamente a través de Webhooks y APIs REST.

2.2 Funciones del Producto

Las principales funcionalidades del sistema agrupan las siguientes capacidades: - **Gestión del Ciclo de Vida de Eventos:** Creación, categorización, publicación y cancelación de eventos con soporte para contenido multimedia y geolocalización. - **Ticketing y Control de Aforo:** Sistema de reservas con bloqueo pesimista en base de datos para evitar sobreventa, controlando capacidades máximas por tipo de ticket. - **Procesamiento de Pagos:** Integración con la pasarela Culqi para transacciones seguras con tarjetas, incluyendo manejo de reembolsos y rechazos. - **Validación de Accesos:** Emisión de códigos QR únicos por ticket y escáner para check-in. - **Interacción Social:** Sistema de calificaciones y comentarios en tiempo real para eventos publicados.

2.3 Clases de Usuario y Características

- **Administrador (admin):** Alta competencia técnica. Responsable de la moderación de la plataforma, gestión global de usuarios, resolución de disputas y configuración de categorías. Uso frecuente.
- **Organizador (organizer):** Competencia técnica media. Productores de eventos que necesitan publicar sus shows, monitorear la venta de entradas en tiempo real y gestionar el escaneo de QR en puerta. Uso diario/semanal.
- **Asistente (registered_user):** Competencia técnica baja/variada. Consumidor final que busca eventos, realiza pagos con tarjeta y presenta su código QR. Uso esporádico o por demanda.

2.4 Entorno Operativo

El sistema operará bajo una arquitectura cliente-servidor nativa web: - **Cliente:** Navegadores web modernos (Chrome, Safari, Firefox, Edge) en dispositivos de escritorio y móviles (diseño responsive con Tailwind CSS). - **Servidor:** Aplicación alojada en contenedores Docker (desplegada vía Kamal), ejecutando Ruby 3.3.1 y Ruby on Rails 8. - **Base de Datos:** PostgreSQL 16 como base de datos relacional principal, manejando Solid Queue para trabajos en segundo plano y Solid Cache.

2.5 Restricciones

- **Restricciones de Diseño:** La aplicación utiliza Hotwire (Turbo/Stimulus), por lo que se restringe el uso de frameworks SPA pesados (como React o Vue) para mantener la renderización del lado del servidor.
- **Restricciones de Negocio:** No se permite la compra de más tickets que el límite configurado por orden (`max_per_order`).
- **Restricciones de Integración:** La comunicación con sistemas externos está restringida a eventos asíncronos vía Webhooks para no degradar el rendimiento (tiempo de respuesta) de la base de datos principal.

2.6 Documentación de Usuario

- **Para Desarrolladores:** Documentación técnica integral disponible en el `README.md` del repositorio y diagramas en la bóveda de Obsidian.
- **Para Organizadores/Asistentes:** Se proveerán guías interactivas y Tooltips dentro de la interfaz de usuario en la aplicación (Fase 2).

2.7 Supuestos y Dependencias

Supuestos: - Los usuarios finales poseen acceso a conexión a internet estable y un dispositivo capaz de leer o mostrar códigos QR. - El volumen de transacciones no excederá los límites actuales configurados en el Rate Limiting durante el primer trimestre de lanzamiento.

Dependencias: - **Culqi API:** Dependencia crítica para la liquidación de fondos y validación de tarjetas. - **Servicios de Geolocalización (OpenStreetMap/Leaflet):** Dependencia para el correcto mapeo y visualización de la dirección de los eventos.

3. Requisitos Funcionales

A continuación se detallan los requisitos funcionales organizados por módulo, representando el estado actual avanzado del sistema.

3.1 Autenticación y Autorización

- **RF-01:** El sistema **DEBE** permitir el registro tradicional validando la cuenta mediante un código numérico de 6 dígitos enviado por email (expira en 15 minutos).
- **RF-02:** El sistema **DEBE** permitir el inicio de sesión y registro automático utilizando Google OAuth 2.0.
- **RF-03:** El sistema **DEBE** restringir el acceso a módulos y acciones basándose en el rol del usuario (Admin, Organizer, Registered User) mediante políticas de Pundit.

3.2 Gestión de Eventos

- **RF-04:** Un usuario con rol **organizer** **DEBE** poder crear, editar y cancelar eventos exclusivamente de su autoría.
- **RF-05:** El sistema **DEBE** permitir cargar imágenes para los eventos y almacenarlas de forma segura utilizando Active Storage.
- **RF-06:** El sistema **DEBE** permitir el ingreso de una dirección física, geocodificar sus coordenadas (lat/lng) y renderizar un mapa interactivo (Leaflet) en el detalle del evento.
- **RF-07:** El sistema **DEBE** proveer un buscador avanzado que permita filtrar eventos por categoría, ciudad, rango de fechas y precio.

3.3 Ticketing y Reservas (Bookings)

- **RF-08:** El sistema **DEBE** asegurar la concurrencia utilizando bloqueos pesimistas en la base de datos para evitar la sobreventa de entradas de un mismo `ticket_type`.
- **RF-09:** Al iniciar la compra, el sistema **DEBE** crear una reserva (`booking`) en estado *pending* y retener la capacidad del evento por un máximo de 15 minutos.
- **RF-10:** El sistema **DEBE** expirar automáticamente las reservas que no hayan sido pagadas superando los 15 minutos, liberando la capacidad nuevamente (vía background jobs).

3.4 Procesamiento de Pagos

- **RF-11:** El sistema **DEBE** procesar cobros integrándose con la API v4 de Culqi, o usar un simulador (`MockGateway`) para entornos locales.
- **RF-12:** El sistema **DEBE** confirmar y marcar la reserva como pagada atómicamente si el cobro es exitoso, impidiendo cobros duplicados.
- **RF-13:** El sistema **DEBE** escuchar notificaciones asíncronas (Webhooks) del proveedor de pagos para gestionar reembolsos y caídas de red, validando el estado a través de una máquina de estados finitos.

3.5 Validación en Puerta (Check-in)

- **RF-14:** El sistema **DEBE** generar y mostrar un código QR único para cada entrada pagada exitosamente.
- **RF-15:** El sistema **DEBERÍA** proveer una vista exclusiva para que el **organizer** pueda escanear los códigos QR y marcar el ticket como consumido, evitando usos duplicados.

3.6 Interacción Social

- **RF-16:** Un asistente autenticado **DEBE** poder crear, leer y eliminar sus propios comentarios en la página de un evento publicado.
- **RF-17:** El sistema **DEBERÍA** permitir a los asistentes añadir eventos a una lista personal de “Favoritos”.

4. Interfaces Externas

4.1 Interfaces de Usuario (UI)

- **Framework:** Tailwind CSS v4 para diseño responsive y utilidades atómicas.
- **Dinamismo:** Hotwire (Turbo Drive, Turbo Frames, Turbo Streams) y Stimulus JS para una experiencia tipo SPA sin comprometer el renderizado en el servidor.
- **Visualización:** Mapas interactivos estilizados mediante Leaflet.
- **Accesibilidad:** La interfaz **DEBERÍA** cumplir con los estándares WCAG 2.1 AA, garantizando el uso de etiquetas ARIA y contrastes adecuados.

4.2 Interfaces de Hardware

- El sistema no interactúa directamente con hardware propietario.
- Los organizadores utilizarán las cámaras de sus propios dispositivos móviles (smartphones o tablets) mediante el navegador web para escanear los códigos QR de las entradas.

4.3 Interfaces de Software (APIs y Servicios)

- **Google OAuth 2.0:** Integración mediante la gema `omniauth-google-oauth2` para delegar la autenticación de usuarios.

- **Culqi API v4:** Pasarela de pagos principal. La comunicación se realiza mediante peticiones REST (JSON) utilizando la llave pública para tokenización en el frontend y la llave secreta para cargos desde el backend.
- **Servicios de Correo (SMTP):** Integración con Gmail u otro proveedor SMTP para el envío de códigos de confirmación de registro (**UserMailer**).
- **OpenStreetMap:** Proveedor de los tiles de mapas utilizados por Leaflet.

4.4 Interfaces de Comunicaciones

- **Protocolos:** Toda comunicación entre el cliente y el servidor, así como entre el servidor y las APIs externas (Culqi, Google), **DEBE** realizarse exclusivamente a través de **HTTPS (TLS 1.2 o superior)**.
- **Webhooks:** Recepción de peticiones POST seguras desde Culqi en el endpoint `/webhooks/payments/receive` para notificar asincrónicamente el resultado de las transacciones (aprobadas, denegadas, reembolsadas).

5. Requisitos No Funcionales

5.1 Rendimiento y Escalabilidad

- **RNF-01:** El sistema **DEBE** soportar transacciones atómicas de compra de entradas (reservas concurrentes) con un tiempo de respuesta de base de datos inferior a 200ms para evitar interbloques masivos.
- **RNF-02:** El sistema **DEBERÍA** manejar al menos 100 usuarios concurrentes intentando comprar entradas para un mismo evento utilizando bloqueos pesimistas (`lock!`).
- **RNF-03:** La renderización de mapas en la vista de detalle de evento **DEBE** cargar en menos de 2 segundos.

5.2 Seguridad y Cumplimiento Legal

- **RNF-04:** Las contraseñas de los usuarios **DEBEN** estar hasheadas con bcrypt (manejado vía Devise).
- **RNF-05:** El sistema **DEBE** proteger las rutas y acciones mediante políticas granulares (Pundit) para asegurar que un **user** no pueda ejecutar acciones de **organizer** o **admin**.
- **RNF-06:** En cumplimiento con la PCI DSS, la aplicación **NUNCA DEBE** almacenar ni transitar números completos de tarjetas de crédito o CVVs en sus propios servidores (tokenización directa en Culqi).
- **RNF-07:** En cumplimiento con la Ley N° 29733 (Perú), los datos personales (nombres, correos) **DEBEN** ser almacenados con controles de acceso estrictos.

5.3 Confiabilidad y Disponibilidad

- **RNF-08:** El sistema **DEBE** asegurar que los trabajos en segundo plano (**Solid Queue**) se ejecuten de forma ininterrumpida para expirar reservas (**ExpireBookingsJob**) cada 5 minutos.
- **RNF-09:** El sistema de pagos **DEBE** contemplar escenarios de caída de red, garantizando el resincronismo del estado del pago a través de Webhooks idempotentes.

5.4 Usabilidad y Accesibilidad

- **RNF-10:** La plataforma **DEBE** ser 100% responsiva, priorizando el enfoque Mobile-First para la compra de tickets y la visualización de códigos QR.
- **RNF-11:** El escáner de validación en puerta para el organizador **DEBE** funcionar en condiciones de poca iluminación utilizando el contraste adecuado.

5.5 Mantenibilidad e Infraestructura

- **RNF-12:** El sistema **DEBE** estar empaquetado en contenedores Docker y desplegado usando Kamal, permitiendo levantamientos sin tiempo de inactividad (zero-downtime deployments).

- **RNF-13:** El sistema **DEBERÍA** mantener una cobertura de pruebas automatizadas (RSpec) superior al 80%, enfocándose críticamente en los Service Objects financieros y de reservas.

6. Otros Requisitos y Apéndices

6.1 Migración de Datos e Internacionalización

- El sistema utiliza la gema `rails-i18n`. Todo el contenido y las etiquetas de la interfaz de usuario **DEBERÍAN** estar preparadas para múltiples idiomas (actualmente enfocado en español).

Apéndice A: Glosario de Estados Centrales

- **Estados de Booking:** `pending` (esperando pago), `confirmed` (pagado), `cancelled` (cancelado manual), `expired` (expiró límite de 15 min).
- **Estados de Pago:** `pending` (procesando), `approved` (aprobado por Culqi), `declined` (tarjeta rechazada), `refunded` (reembolsado).

Apéndice B: Lista de Problemas Pendientes / Riesgos (Risk Register)

- **Riesgo 1:** Latencia en la recepción del Webhook de Culqi podría causar que una reserva expire localmente antes de registrar el pago exitoso. **Mitigación:** La máquina de estados permite reconciliación y el Job de expiración verifica si existe un pago en estado pendiente antes de borrar.
- **Riesgo 2:** Los usuarios pueden perder conectividad durante el checkout en sus móviles. **Mitigación:** La vista de checkout debe manejar errores de red y permitir reintentos dentro de la ventana de 15 minutos.

Estado del Proyecto: event-management

¡Felicitaciones! La fase de elicitación base se ha completado al 100%.

Se han documentado: - 1.0 Introducción, Propósito, Convenciones y Alcance. - 2.0 Descripción General y Perfiles. - 3.0 Requisitos Funcionales (RF-01 a RF-17). - 4.0 Interfaces Externas (Culqi, Google OAuth). - 5.0 Requisitos No Funcionales (PCI DSS, Escalabilidad, Solid Queue). - 6.0 Otros Requisitos y Riesgos.

El documento ahora actúa como base viva para el desarrollo y las pruebas.